Object-Oriented Programming

Complete Reference Guide

C++ & Java • Full Runnable Examples

All Pillars • Exceptions • Advanced Concepts • Cheat Sheet

Compiled for SE Students • Spring 2026

Riphah International University

Object-Oriented Programming

Complete Reference Guide

C++ & Java • Full Runnable Examples

All Pillars • Exceptions • Advanced Concepts • Cheat Sheet

Compiled for SE Students • Spring 2026

Riphah International University

Table of Contents

1. The Four Pillars of OOP

Object-Oriented Programming is a paradigm built around **objects** — bundles of data (attributes) and behaviour (methods). Four core principles define OOP:

Pillar	One-Line Definition
Encapsulation	Wrapping data and methods together; hiding internal state.
Abstraction	Exposing only what is necessary; hiding complexity.
Inheritance	Deriving new classes from existing ones to reuse behaviour.
Polymorphism	Same interface, different implementations — one name, many forms.

1.1 Encapsulation

Encapsulation bundles data and the methods that operate on it into a single unit (class), and restricts direct access to some components. This protects object integrity by preventing outsiders from setting data to an invalid state.

- Access modifiers control visibility: **private**, **protected**, **public**.
- Getters/setters provide controlled access to private fields.

C++

```
#include <iostream>
#include <string>
using namespace std;
class BankAccount {
private:
string owner;
double balance; // hidden from outside
public:
// Constructor
BankAccount(string name, double initial)
: owner(name), balance(initial) {}
// Getter
double getBalance() const { return balance; }
// Controlled mutation
void deposit(double amount) {
if (amount > 0) balance += amount;
}
void withdraw(double amount) {
if (amount > 0 && amount <= balance)
balance -= amount;
else
```

```
cout << "Invalid withdrawal\n";
}
void display() const {
cout << owner << " | Balance: $"
<< balance << "\n";
}
};
int main() {
BankAccount acc("Mohid", 1000.0);
acc.deposit(500);
acc.withdraw(200);
acc.display(); // Mohid | Balance: $1300
// acc.balance = -999; // ERROR: private!
return 0;
}
```

Java

```
public class BankAccount {
private String owner;
private double balance; // hidden
public BankAccount(String name,
double initial) {
this.owner = name;
this.balance = initial;
}
// Getter
public double getBalance() {
return balance;
}
// Controlled mutation
public void deposit(double amount) {
if (amount > 0) balance += amount;
}
public void withdraw(double amount) {
if (amount > 0 && amount <= balance)
balance -= amount;
else
System.out.println(
"Invalid withdrawal");
}
public void display() {
System.out.println(owner
+ " | Balance: $" + balance);
}
public static void main(String[] args) {
BankAccount acc =
```

```
new BankAccount("Mohid", 1000.0);
acc.deposit(500);
acc.withdraw(200);
acc.display(); // Mohid | $1300.0
// acc.balance = -999; // ERROR
}
}
```

1.2 Abstraction

Abstraction means showing only the relevant details of an object and hiding the implementation complexity. You interact with a car's steering wheel without knowing the power-steering mechanism.

- In C++: achieved via abstract classes (pure virtual functions).
- In Java: achieved via abstract classes AND interfaces.

C++

```
#include <iostream>
using namespace std;
// Abstract class — cannot be instantiated
class Shape {
public:
virtual double area() = 0; // pure virtual
virtual void draw() = 0;
virtual ~Shape() {}
};
class Circle : public Shape {
double r;
public:
Circle(double radius) : r(radius) {}
double area() override {
return 3.14159 * r * r;
}
void draw() override {
cout << "Drawing Circle, area = "
<< area() << "\n";
}
};
class Rectangle : public Shape {
double w, h;
public:
Rectangle(double w, double h)
: w(w), h(h) {}
double area() override { return w * h; }
void draw() override {
cout << "Drawing Rect, area = "
```

```
<< area() << "\n";
}
};
int main() {
Shape* s1 = new Circle(5);
Shape* s2 = new Rectangle(4, 6);
s1->draw(); // Drawing Circle, area=78.54
s2->draw(); // Drawing Rect, area=24
delete s1; delete s2;
return 0;
}
```

Java

```
abstract class Shape {
public abstract double area();
public abstract void draw();
}
class Circle extends Shape {
private double r;
public Circle(double r) { this.r = r; }
@Override
public double area() {
return Math.PI * r * r;
}
@Override
public void draw() {
System.out.printf(
"Drawing Circle, area=%.2f%n",
area());
}
}
class Rectangle extends Shape {
private double w, h;
public Rectangle(double w, double h) {
this.w = w; this.h = h;
}
@Override
public double area() { return w * h; }
@Override
public void draw() {
System.out.printf(
"Drawing Rect, area=%.2f%n",
area());
}
}
public class Main {
```

```
public static void main(String[] args) {
Shape s1 = new Circle(5);
Shape s2 = new Rectangle(4, 6);
s1.draw(); // Circle area=78.54
s2.draw(); // Rect area=24.00
}
}
```

1.3 Inheritance

Inheritance lets a class (child/derived) acquire properties and methods of another class (parent/base). It promotes code reuse and establishes an IS-A relationship.

- C++ supports **single, multiple, multilevel, hierarchical,** and **hybrid** inheritance.
- Java supports **single and multilevel** inheritance for classes; multiple inheritance is achieved through **interfaces**.

C++

```
#include <iostream>
using namespace std;
class Animal { // Base class
protected:
string name;
public:
Animal(string n) : name(n) {}
void breathe() {
cout << name << " breathes.\n";
}
virtual void speak() { // overridable
cout << name << " makes a sound.\n";
}
};
class Dog : public Animal { // Single inheritance
public:
Dog(string n) : Animal(n) {}
void speak() override {
cout << name << " says: Woof!\n";
}
void fetch() {
cout << name << " fetches the ball.\n";
}
};
class GuideDog : public Dog { // Multilevel
public:
GuideDog(string n) : Dog(n) {}
void guide() {
```

```
cout << name << " guides its owner.\n";
}
};
int main() {
GuideDog gd("Rex");
gd.breathe(); // from Animal
gd.speak(); // from Dog (overridden)
gd.fetch(); // from Dog
gd.guide(); // own method
return 0;
}
```

Java

```
class Animal {
protected String name;
public Animal(String n) { this.name = n; }
public void breathe() {
System.out.println(name + " breathes.");
}
public void speak() {
System.out.println(
name + " makes a sound.");
}
}
class Dog extends Animal {
public Dog(String n) { super(n); }
@Override
public void speak() {
System.out.println(
name + " says: Woof!");
}
public void fetch() {
System.out.println(
name + " fetches the ball.");
}
}
class GuideDog extends Dog {
public GuideDog(String n) { super(n); }
public void guide() {
System.out.println(
name + " guides its owner.");
}
}
public class Main {
public static void main(String[] args) {
GuideDog gd = new GuideDog("Rex");
```

```
gd.breathe(); // from Animal
gd.speak(); // Dog's override
gd.fetch(); // from Dog
gd.guide(); // own method
}
}
```

1.4 Polymorphism

Polymorphism (Greek: 'many forms') allows the same interface to be used for different underlying data types. Two types:

- **Compile-time (static)** — function overloading, operator overloading; resolved at compile time.
- **Run-time (dynamic)** — method overriding via virtual functions / inheritance; resolved at runtime.

C++

```
#include <iostream>
using namespace std;
// Compile-time: function overloading
class Printer {
public:
void print(int x) { cout << "Int: " << x << "\n"; }
void print(double x) { cout << "Double: " << x << "\n"; }
void print(string x) { cout << "String: " << x << "\n"; }
};
// Run-time: virtual functions
class Vehicle {
public:
virtual void fuelType() {
cout << "Generic fuel\n";
}
virtual ~Vehicle() {}
};
class Car : public Vehicle {
public: void fuelType() override { cout << "Petrol\n"; }
};
class ElecCar : public Vehicle {
public: void fuelType() override { cout << "Electric\n"; }
};
int main() {
// Compile-time
Printer p;
p.print(42);
p.print(3.14);
p.print("Hello");
// Run-time via base pointer
```

```
Vehicle* v1 = new Car();
Vehicle* v2 = new ElecCar();
v1->fuelType(); // Petrol
v2->fuelType(); // Electric
delete v1; delete v2;
return 0;
}
```

Java

```
public class PolymorphismDemo {
// Compile-time: method overloading
static void print(int x) {
System.out.println("Int: " + x);
}
static void print(double x) {
System.out.println("Double: " + x);
}
static void print(String x) {
System.out.println("String: " + x);
}
// Run-time: method overriding
static class Vehicle {
public void fuelType() {
System.out.println("Generic fuel");
}
}
static class Car extends Vehicle {
@Override public void fuelType() {
System.out.println("Petrol");
}
}
static class ElecCar extends Vehicle {
@Override public void fuelType() {
System.out.println("Electric");
}
}
public static void main(String[] args) {
// Compile-time
print(42); print(3.14); print("Hi");
// Run-time via base reference
Vehicle v1 = new Car();
Vehicle v2 = new ElecCar();
v1.fuelType(); // Petrol
v2.fuelType(); // Electric
}
}
```

2. Constructors & Destructors

A **constructor** is a special method called automatically when an object is created. A **destructor** (C++ only) is called when the object goes out of scope or is deleted. Java uses garbage collection instead.

Type	C++	Java
Default Ctor	Compiler-generated if none defined	Compiler-generated if none defined
Parameterized Ctor	Yes	Yes
Copy Constructor	Yes (deep/shallow)	No — use clone() / copy factory
Move Constructor	Yes (C++11)	No
Destructor	~ClassName()	finalize() — deprecated; use try-with-resources

C++

```
#include <iostream>
#include <cstring>
using namespace std;
class Student {
char* name;
int rollNo;
public:
// Default constructor
Student() : name(nullptr), rollNo(0) {
cout << "Default ctor called\n";
}
// Parameterized constructor
Student(const char* n, int r) : rollNo(r) {
name = new char[strlen(n)+1];
strcpy(name, n);
cout << "Param ctor: " << name << "\n";
}
// Copy constructor (deep copy)
Student(const Student& other) : rollNo(other.rollNo) {
name = new char[strlen(other.name)+1];
strcpy(name, other.name);
cout << "Copy ctor: " << name << "\n";
}
// Destructor
~Student() {
cout << "Destructor: " << (name?name:"null") << "\n";
delete[] name;
}
void display() const {
cout << name << " #" << rollNo << "\n";
```

```
}
};
int main() {
Student s1("Mohid", 101);
Student s2 = s1; // copy ctor
s1.display();
s2.display();
return 0; // destructors called here
}
```

Java

```
public class Student {
private String name;
private int rollNo;
// Default constructor
public Student() {
System.out.println("Default ctor");
}
// Parameterized constructor
public Student(String name, int rollNo) {
this.name = name;
this.rollNo = rollNo;
System.out.println(
"Param ctor: " + name);
}
// Copy constructor (manual)
public Student(Student other) {
this.name = other.name; // Strings immutable
this.rollNo = other.rollNo;
System.out.println(
"Copy ctor: " + name);
}
// Java has no real destructor.
// Use try-with-resources for cleanup.
@Override
protected void finalize() {
// Deprecated — don't rely on this
System.out.println(
"GC collecting: " + name);
}
public void display() {
System.out.println(
name + " #" + rollNo);
}
public static void main(String[] args) {
Student s1 = new Student("Mohid",101);
```

```
Student s2 = new Student(s1); // copy
s1.display();
s2.display();
}
}
```

3. Static vs Non-Static Members

A **static** member belongs to the *class itself*, not to any individual object. All objects share the same static member. A **non-static** (instance) member belongs to a specific object.

- Static variables: one copy per class, persists for the program lifetime.
- Static methods: can be called without creating an object; cannot access 'this'.
- Non-static: each object gets its own copy; can use 'this'.

C++

```
#include <iostream>
using namespace std;
class Counter {
public:
static int count; // static — shared
int id; // non-static — per object
Counter() {
++count;
id = count;
cout << "Object #" << id << " created\n";
}
// Static method
static void showCount() {
cout << "Total objects: " << count << "\n";
}
// Non-static method
void showId() const {
cout << "My id = " << id << "\n";
}
};
// Static member must be defined outside
int Counter::count = 0;
int main() {
Counter::showCount(); // 0 — no object needed
Counter a, b, c;
Counter::showCount(); // 3
a.showId(); // My id = 1
b.showId(); // My id = 2
return 0;
}
```

Java

```
public class Counter {
static int count = 0; // shared across all
int id; // per-object
public Counter() {
```

```
count++;
id = count;
System.out.println(
"Object #" + id + " created");
}
// Static method
public static void showCount() {
System.out.println(
"Total objects: " + count);
}
// Non-static method
public void showId() {
System.out.println("My id = " + id);
}
public static void main(String[] args) {
Counter.showCount(); // 0
Counter a = new Counter();
Counter b = new Counter();
Counter c = new Counter();
Counter.showCount(); // 3
a.showId(); // My id = 1
b.showId(); // My id = 2
}
}
```

■ *In C++, a static method cannot access non-static members because there is no 'this' pointer. Same rule applies in Java.*

4. Friend Functions (C++) & Package Access (Java)

A **friend function** in C++ is a non-member function granted access to the private and protected members of a class. Java has no direct equivalent; it achieves similar effects through package-private access (default access modifier) or inner classes.

C++

```
#include <iostream>
using namespace std;
class Box {
private:
double length;
double width;
double height;
public:
Box(double l, double w, double h)
: length(l), width(w), height(h) {}
// Declare friend function
friend double volume(const Box& b);
// Friend class (all methods of Printer
// can access Box's private members)
friend class Printer;
};
// Friend function — not a member of Box
double volume(const Box& b) {
return b.length * b.width * b.height;
}
class Printer {
public:
void printBox(const Box& b) {
cout << "L=" << b.length
<< " W=" << b.width
<< " H=" << b.height << "\n";
}
};
int main() {
Box b(3, 4, 5);
cout << "Volume = " << volume(b) << "\n"; // 60
Printer p;
p.printBox(b);
return 0;
}
```

Java

```
// Java has NO friend keyword.
// Package-private (default access) gives
// access to all classes in same package.
// File: Box.java (package geometry)
package geometry;
class Box { // package-private class
double length; // package-private fields
double width;
double height;
Box(double l, double w, double h) {
length = l; width = w; height = h;
}
}
// File: VolumeUtil.java (same package)
package geometry;
class VolumeUtil {
static double volume(Box b) {
// Can access b.length, etc.
// because same package
return b.length * b.width * b.height;
}
}
// File: Main.java (same package)
package geometry;
public class Main {
public static void main(String[] args) {
Box b = new Box(3, 4, 5);
System.out.println(
"Volume = " + VolumeUtil.volume(b));
}
}
```

■ ***C++ Note*** Friend relationships are not inherited. If B is a friend of A, B's subclasses are NOT friends of A. Use friend sparingly — it breaks encapsulation.

5. Inline Functions (C++) & JIT Inlining (Java)

The **inline** keyword in C++ requests the compiler to insert the function body directly at every call site, eliminating function-call overhead. The compiler may ignore the hint for large functions. Java's JIT (Just-In-Time) compiler performs inlining automatically — the programmer has no explicit control.

C++

```
#include <iostream>
using namespace std;
// Explicit inline request
inline int square(int x) {
return x * x;
}
// Methods defined inside class body
// are implicitly inline
class MathHelper {
public:
inline double cube(double x) {
return x * x * x;
}
// Larger functions should NOT be inline
void heavyComputation(int n);
};
// Defined outside — NOT inline by default
void MathHelper::heavyComputation(int n) {
// ... lots of code ...
cout << "Heavy: " << n << "\n";
}
int main() {
cout << square(5) << "\n"; // 25
MathHelper m;
cout << m.cube(3) << "\n"; // 27
m.heavyComputation(10);
return 0;
}
```

Java

```
// Java has NO inline keyword.
// The JVM's JIT compiler decides automatically.
public class MathHelper {
// Small method — JIT will likely inline this
public static int square(int x) {
return x * x;
}
public static double cube(double x) {
```

```
return x * x * x;
}
// final methods are better inlining candidates
// because JIT knows they won't be overridden
public final int add(int a, int b) {
return a + b;
}
public static void main(String[] args) {
System.out.println(square(5)); // 25
System.out.println(cube(3)); // 27.0
MathHelper m = new MathHelper();
System.out.println(m.add(3,4));// 7
}
}
```

■ *In C++, inline is only a hint. Modern compilers (g++, clang++) inline functions based on their own heuristics regardless of the keyword. Java's JIT similarly ignores programmer intent.*

6. Scope Resolution Operator (C++)

The **scope resolution operator** :: in C++ is used to:

- Define a member function outside its class body.
- Access global variables when a local variable has the same name.
- Access static members of a class.
- Specify which base class's member to use in case of ambiguity.
- Access namespaced entities (e.g., std::cout).

Java has no scope resolution operator. Java uses the **dot operator** for everything, and **super** to access parent class members.

C++

```
#include <iostream>
using namespace std;
int x = 100; // global
namespace MathLib {
double PI = 3.14159;
double circleArea(double r);
}
// Define namespace function with ::
double MathLib::circleArea(double r) {
return MathLib::PI * r * r;
}
class Engine {
public:
static int horsepower;
void start(); // declared
void stop(); // declared
};
int Engine::horsepower = 250; // static init
// Defined outside with ::
void Engine::start() {
cout << "Engine started. HP="
<< Engine::horsepower << "\n";
}
void Engine::stop() {
cout << "Engine stopped.\n";
}
int main() {
int x = 5; // local x shadows global
cout << "Local x = " << x << "\n"; // 5
cout << "Global x = " << ::x << "\n"; // 100
cout << MathLib::circleArea(3) << "\n"; // 28.27
Engine e;
```

```
e.start();
cout << Engine::horsepower << "\n"; // 250
e.stop();
return 0;
}
```

Java

```
// Java uses dot (.) for member access
// and 'super' for parent members.
class Vehicle {
int speed = 60;
public void describe() {
System.out.println(
"Vehicle speed: " + speed);
}
}
class Car extends Vehicle {
int speed = 120; // shadows parent's speed
@Override
public void describe() {
// 'this.speed' — own field
System.out.println(
"Car speed: " + this.speed);
// 'super.speed' — parent's field
System.out.println(
"Parent speed: " + super.speed);
// Call parent method
super.describe();
}
// Accessing static members via class name
static int maxSpeed = 200;
public static void showMax() {
System.out.println(
"Max: " + Car.maxSpeed);
}
}
public class Main {
public static void main(String[] args) {
Car c = new Car();
c.describe();
Car.showMax(); // static via class name
}
}
```

7. Operator Overloading

Operator overloading lets you redefine what operators like +, -, ==, << do for user-defined types. C++ supports extensive operator overloading. Java does **NOT** support operator overloading (except built-in + for String concatenation).

C++

```
#include <iostream>
using namespace std;
class Vector2D {
public:
double x, y;
Vector2D(double x=0, double y=0)
: x(x), y(y) {}
// Overload +
Vector2D operator+(const Vector2D& other) const {
return Vector2D(x + other.x,
y + other.y);
}
// Overload -
Vector2D operator-(const Vector2D& other) const {
return Vector2D(x - other.x,
y - other.y);
}
// Overload ==
bool operator==(const Vector2D& o) const {
return x==o.x && y==o.y;
}
// Overload << (friend — needs stream access)
friend ostream& operator<<(ostream& os,
const Vector2D& v){
return os << "(" << v.x << ", "
<< v.y << ")";
}
};
int main() {
Vector2D v1(1, 2), v2(3, 4);
cout << v1 + v2 << "\n"; // (4, 6)
cout << v2 - v1 << "\n"; // (2, 2)
cout << (v1 == v2) << "\n"; // 0 (false)
return 0;
}
```

Java

```
// Java does NOT support operator overloading.
```

```
// You use named methods instead.
public class Vector2D {
public double x, y;
public Vector2D(double x, double y) {
this.x = x; this.y = y;
}
// 'add' replaces operator+
public Vector2D add(Vector2D other) {
return new Vector2D(
this.x + other.x,
this.y + other.y);
}
// 'subtract' replaces operator-
public Vector2D subtract(Vector2D other) {
return new Vector2D(
this.x - other.x,
this.y - other.y);
}
// equals() replaces operator==
@Override
public boolean equals(Object obj) {
if (!(obj instanceof Vector2D)) return false;
Vector2D o = (Vector2D) obj;
return this.x == o.x && this.y == o.y;
}
// toString() replaces operator<<
@Override
public String toString() {
return "(" + x + ", " + y + ")";
}
public static void main(String[] args) {
Vector2D v1 = new Vector2D(1,2);
Vector2D v2 = new Vector2D(3,4);
System.out.println(v1.add(v2)); // (4,6)
System.out.println(v2.subtract(v1)); // (2,2)
System.out.println(v1.equals(v2)); // false
}
}
```

■ ***Java Note*** Java's + operator is automatically overloaded for String concatenation by the compiler — this is the ONLY built-in overloading in Java. You cannot define your own.

8. Abstract Classes & Interfaces

An **abstract class** is a class that cannot be instantiated and may contain abstract (unimplemented) methods that subclasses must implement. An **interface** is a pure contract — in Java it defines what a class must do; in C++ there is no interface keyword, but a class with all pure virtual functions serves the same purpose.

Feature	Abstract Class	Interface (Java)
Instantiation	No	No
Concrete methods	Yes	Yes (default/static in Java 8+)
Fields / State	Yes	Only constants (public static final)
Constructor	Yes	No
Multiple inheritance	No (Java) / Yes (C++)	Yes — a class implements many
Access modifiers	Any	public (implicit)

C++

```
#include <iostream>
using namespace std;
// Abstract class in C++
class Drawable {
public:
virtual void draw() = 0; // pure virtual
virtual void resize() = 0;
virtual ~Drawable() {}
// Can have concrete methods too:
void log() { cout << "Drawing...\n"; }
};
// Another abstract class (simulates interface)
class Saveable {
public:
virtual void save() = 0;
virtual ~Saveable() {}
};
// Multiple inheritance of abstract classes
class Document : public Drawable,
public Saveable {
public:
void draw() override { cout << "Render doc\n"; }
void resize() override { cout << "Resize doc\n"; }
void save() override { cout << "Save doc\n"; }
};
int main() {
```

```
// Drawable* d = new Drawable(); // ERROR!
Document doc;
doc.log(); // concrete from Drawable
doc.draw();
doc.resize();
doc.save();
return 0;
}
```

Java

```
// Abstract class
abstract class Drawable {
public abstract void draw();
public abstract void resize();
// Concrete method
public void log() {
System.out.println("Drawing...");
}
}
// Interface — pure contract
interface Saveable {
void save(); // implicitly abstract
// Default method (Java 8+)
default void autoSave() {
System.out.println("Auto-saving...");
}
}
interface Printable {
void print();
}
// Class extends one abstract class
// and implements multiple interfaces
class Document extends Drawable
implements Saveable, Printable {
@Override public void draw() {
System.out.println("Render doc");
}
@Override public void resize() {
System.out.println("Resize doc");
}
@Override public void save() {
System.out.println("Save doc");
}
@Override public void print() {
System.out.println("Print doc");
}
```

```
public static void main(String[] args) {
Document doc = new Document();
doc.log(); // from Drawable
doc.draw();
doc.autoSave(); // default from Saveable
doc.save();
doc.print();
}
}
```

9. Virtual Functions & Polymorphism (vtable)

When a base class pointer points to a derived object, C++ by default calls the **base class** version of a function. To enable dynamic dispatch (calling the derived version), the function must be declared **virtual**.

How vtable works

Every class with virtual functions has a hidden **vtable** (virtual function table) — an array of function pointers. Each object holds a hidden **vptr** (virtual pointer) pointing to its class's vtable. At runtime, the call goes through vptr → vtable → correct function.

Animal vtable	Dog vtable
`speak → Animal::speak`	`speak → Dog::speak`
`~Animal`	`~Dog`

C++

```
#include <iostream>
using namespace std;
class Animal {
public:
// WITHOUT virtual: early binding (compile-time)
// WITH virtual: late binding (run-time)
virtual void speak() {
cout << "Animal speaks\n";
}
// Virtual destructor — ALWAYS use when
// deleting derived objects via base pointer
virtual ~Animal() {
cout << "~Animal\n";
}
};
class Dog : public Animal {
public:
void speak() override {
cout << "Dog: Woof!\n";
}
~Dog() override { cout << "~Dog\n"; }
};
class Cat : public Animal {
public:
void speak() override {
cout << "Cat: Meow!\n";
}
};
```

```
int main() {
Animal* a1 = new Dog();
Animal* a2 = new Cat();
a1->speak(); // Dog: Woof! (vtable dispatch)
a2->speak(); // Cat: Meow!
delete a1; // ~Dog then ~Animal (correct!)
delete a2;
// Without virtual destructor:
// only ~Animal would be called — UNDEFINED BEHAVIOUR
return 0;
}
```

Java

```
// ALL non-static methods in Java are
// virtual by default.
// Use 'final' to prevent overriding.
class Animal {
public void speak() {
System.out.println("Animal speaks");
}
}
class Dog extends Animal {
@Override
public void speak() {
System.out.println("Dog: Woof!");
}
}
class Cat extends Animal {
@Override
public void speak() {
System.out.println("Cat: Meow!");
}
}
public class Main {
public static void main(String[] args) {
Animal a1 = new Dog();
Animal a2 = new Cat();
a1.speak(); // Dog: Woof! (dynamic dispatch)
a2.speak(); // Cat: Meow!
// Java has no manual delete.
// GC handles memory.
// 'final' prevents override:
// final void speak() {} ← can't override
}
}
```

■ *C++ Warning*

Always declare the destructor virtual in base classes when using polymorphism. Without it, deleting a derived object via a base pointer causes undefined behaviour — the derived destructor never runs, causing memory leaks.

10. Templates (C++) & Generics (Java)

Both allow writing type-independent code. C++ **templates** are resolved at compile time (code is generated for each type used). Java **generics** use **type erasure** — the generic type information is removed at compile time and replaced with Object (or the bound type), so there is only one class at runtime.

C++

```
#include <iostream>
#include <vector>
using namespace std;
// Function template
template <typename T>
T maxOf(T a, T b) {
return (a > b) ? a : b;
}
// Class template
template <typename T>
class Stack {
vector<T> data;
public:
void push(T val) { data.push_back(val); }
T pop() {
T top = data.back();
data.pop_back();
return top;
}
bool empty() const { return data.empty(); }
int size() const { return data.size(); }
};
// Template specialisation
template <>
const char* maxOf(const char* a, const char* b) {
return (strcmp(a, b) > 0) ? a : b;
}
int main() {
cout << maxOf(10, 20) << "\n"; // 20
cout << maxOf(3.5, 2.1) << "\n"; // 3.5
Stack<int> si;
si.push(1); si.push(2); si.push(3);
cout << si.pop() << "\n"; // 3
Stack<string> ss;
ss.push("Hello"); ss.push("World");
cout << ss.pop() << "\n"; // World
return 0;
```

```
}
```

Java

```
import java.util.ArrayList;
// Generic method
class Util {
public static <T extends Comparable<T>>
T maxOf(T a, T b) {
return (a.compareTo(b) > 0) ? a : b;
}
}
// Generic class
class Stack<T> {
private ArrayList<T> data = new ArrayList<>();
public void push(T val) { data.add(val); }
public T pop() {
return data.remove(data.size() - 1);
}
public boolean empty() {
return data.isEmpty();
}
public int size() { return data.size(); }
}
// Bounded type parameter
class MathBox<T extends Number> {
private T value;
public MathBox(T v) { value = v; }
public double doubled() {
return value.doubleValue() * 2;
}
}
public class Main {
public static void main(String[] args) {
System.out.println(
Util.maxOf(10, 20)); // 20
System.out.println(
Util.maxOf(3.5, 2.1)); // 3.5
Stack<Integer> si = new Stack<>();
si.push(1); si.push(2); si.push(3);
System.out.println(si.pop()); // 3
MathBox<Integer> mb = new MathBox<>(7);
System.out.println(mb.doubled()); // 14.0
}
}
```

11. Exception Handling & the Diamond Problem

11.1 Exception Handling

Both C++ and Java use **try-catch-finally/throw** for exception handling. Key differences:

- C++: Can throw any type (int, string, class). No checked exceptions.
- Java: Only Throwable subclasses. Has checked exceptions that must be declared or caught.
- Java: finally block always runs; C++ uses RAII / destructors for cleanup.

C++

```
#include <iostream>
#include <stdexcept>
using namespace std;
// Custom exception class
class InsufficientFundsException
: public runtime_error {
double amount;
public:
InsufficientFundsException(double a)
: runtime_error("Insufficient funds"),
amount(a) {}
double getAmount() const { return amount; }
};
class Account {
double balance;
public:
Account(double b) : balance(b) {}
void withdraw(double amt) {
if (amt > balance)
throw InsufficientFundsException(amt);
balance -= amt;
cout << "Withdrew $" << amt << "\n";
}
};
int main() {
Account acc(500.0);
try {
acc.withdraw(200);
acc.withdraw(400); // triggers exception
}
catch (const InsufficientFundsException& e) {
cout << "Error: " << e.what()
<< " | Tried: $"
```

```
<< e.getAmount() << "\n";
}
catch (const exception& e) {
cout << "Generic: " << e.what() << "\n";
}
// No finally in C++ — use RAII/destructors
cout << "Program continues\n";
return 0;
}
```

Java

```
// Custom checked exception
class InsufficientFundsException
extends Exception {
private double amount;
public InsufficientFundsException(double a) {
super("Insufficient funds");
this.amount = a;
}
public double getAmount() { return amount; }
}
class Account {
private double balance;
public Account(double b) { balance = b; }
// Must declare checked exception
public void withdraw(double amt)
throws InsufficientFundsException {
if (amt > balance)
throw new InsufficientFundsException(amt);
balance -= amt;
System.out.println("Withdrew $" + amt);
}
}
public class Main {
public static void main(String[] args) {
Account acc = new Account(500.0);
try {
acc.withdraw(200);
acc.withdraw(400); // exception
} catch (InsufficientFundsException e) {
System.out.println("Error: "
+ e.getMessage()
+ " | Tried: $" + e.getAmount());
} finally {
// Always runs — cleanup here
System.out.println("Finally block");
}
```

```
System.out.println("Program continues");
}
}
```

11.2 The Diamond Problem (C++)

The **diamond problem** occurs in multiple inheritance when two parent classes share a common grandparent. Without care, the derived class gets **two copies** of the grandparent's data, causing ambiguity.

C++

```
#include <iostream>
using namespace std;
class Animal {
public:
int age = 5;
void breathe() { cout << "Breathing\n"; }
};
// WITHOUT virtual: Dog and Cat each get
// their own copy of Animal's members
class Dog : virtual public Animal { }; // <-- 'virtual'
class Cat : virtual public Animal { }; // <-- 'virtual'
// CatDog now has only ONE copy of Animal
class CatDog : public Dog, public Cat {
public:
void speak() {
cout << "Woof-Meow! Age=" << age << "\n";
breathe(); // no ambiguity now
}
};
int main() {
CatDog cd;
cd.speak();
// Without 'virtual' above:
// cd.age; // COMPILE ERROR — ambiguous
// cd.breathe(); // COMPILE ERROR — ambiguous
return 0;
}
```

```
/* Without virtual inheritance, you'd get:
error: 'age' is ambiguous
because CatDog has two 'Animal' sub-objects:
CatDog::Dog::Animal::age
CatDog::Cat::Animal::age
*/
```

Java

```
// Java DOES NOT have the diamond problem
// for classes — single class inheritance only.
// But interfaces CAN have a diamond:
interface A { default void hello() {
System.out.println("A"); } }
interface B extends A { default void hello() {
System.out.println("B"); } }
interface C extends A { default void hello() {
System.out.println("C"); } }
// Class must RESOLVE the conflict explicitly
class D implements B, C {
@Override
public void hello() {
B.super.hello(); // explicitly choose B
// or C.super.hello() for C
// or write your own body
System.out.println("D resolved it");
}
public static void main(String[] args) {
D d = new D();
d.hello();
// Output:
// B
// D resolved it
}
}
```

■ ***Diamond Problem Summary***	C++: Use 'virtual' inheritance to ensure only ONE copy of the grandparent. Java: No problem for classes (single inheritance). For interfaces with default methods, the implementing class MUST override and resolve ambiguity manually.

12. Quick-Reference Cheat Sheet

A condensed reference for all major OOP concepts covered in this guide.

Concept	C++	Java
Encapsulation	private/protected + getters/setters	Same — private fields + public getters/setters
Abstraction	Abstract class with pure virtual (= 0)	abstract class / interface keyword
Inheritance	class D : public B { }	class D extends B { }
Polymorphism	virtual + override keywords	All methods virtual by default; use @Override
Constructor	ClassName() { } or init list	ClassName() { } or field-init list
Destructor	~ClassName() { delete...; }	finalize() deprecated; use try-with-resources
Static	static int x; ClassName::x	static int x; ClassName.x
Friend	friend void func(ClassName&);	No equivalent — use package-private or inner class
Inline	inline returnType func() { }	JIT decides automatically; use final for hints
Scope ::	ClassName::member, ::global	Use dot (.) and super keyword instead
Op. Overload	Type operator+(const Type& o) { }	Not supported — use named methods (add, equals)
Abstract Class	class A { virtual void f()=0; };	abstract class A { abstract void f(); }
Interface	class I { virtual void f()=0; }; (pure abstract)	interface I { void f(); }
vtable/dispatch	virtual keyword enables late binding	Late binding is default for all non-final methods
Templates	template<typename T> class Stack { };	class Stack<T> { } — type erasure at runtime
this pointer	this->field; return *this;	this.field; return this;
Poly w/ ptr	Base* p = new Derived();	Base b = new Derived();
const member fn	void f() const {}	No equiv — use final method
const/final var	const int x = 5;	final int x = 5;
final class	No keyword (pattern)	final class MyClass {}
Shallow copy	Default copy ctor (danger)	super.clone() shares refs
Deep copy	Custom copy ctor + new[]	.clone() with new array
pub inherit	class D : public B	class D extends B (only mode)
priv inherit	class D : private B	Use composition instead
Diamond Fix	virtual public BaseClass	Must @Override default method; compiler forces it
Exception	throw MyEx(); try { } catch(MyEx& e) { }	throw new MyEx(); try { } catch(MyEx e) { } finally

End of OOP Reference Guide — C++ & Java | Spring 2026 | Riphah International University

13. The `this` Pointer & Polymorphism with Pointers

13.1 The `this` Pointer

Inside every non-static member function, **this** is an implicit pointer to the object that invoked the function. In Java, **this** is a reference (not a raw pointer) to the current object.

- **Use 1:** Resolve name conflicts between a parameter and a member field.
- **Use 2:** Return the current object (return *this in C++, return this in Java) for method chaining.
- **Use 3:** Pass the current object to another function or store self-reference.

C++

```
#include
using namespace std;
class Builder {
string name;
int age;
public:
// Use 1: resolve name conflict
Builder(string name, int age) {
this->name = name; // this->name = member
this->age = age; // age (right) = param
}
// Use 2: return *this for method chaining
Builder& setName(string name) {
this->name = name;
return *this;
}
Builder& setAge(int age) {
this->age = age;
return *this;
}
void display() const {
cout << name << ", age " << age << "\n";
}
};
int main() {
Builder b("", 0);
// Method chaining via *this
b.setName("Mohid").setAge(20).display();
// Output: Mohid, age 20
return 0;
}
```

Java

```
public class Builder {
private String name;
private int age;
// Use 1: resolve name conflict
public Builder(String name, int age) {
this.name = name; // this.name = field
this.age = age;
}
// Use 2: return this for method chaining
public Builder setName(String name) {
this.name = name;
return this;
}
public Builder setAge(int age) {
this.age = age;
return this;
}
public void display() {
System.out.println(name + ", age " + age);
}
public static void main(String[] args) {
Builder b = new Builder("", 0);
b.setName("Mohid")
.setAge(20)
.display();
// Output: Mohid, age 20
}
}
```

13.2 Polymorphism with Pointers / References

Runtime polymorphism REQUIRES indirection. In C++ you need a **base class pointer (Base*) or reference (Base&)** to a derived object. Calling through a value (not pointer) causes object slicing. In Java, all object variables are already references, so dynamic dispatch is always on.

■ ***Object Slicing (C++)***	Assigning a Derived object to a Base variable (not pointer/reference) cuts off the derived part. Only the Base sub-object is copied. speak() then calls the Base version. Always use Base* or Base& for polymorphism.

C++

```
#include
using namespace std;
class Animal {
```

```
public:
string name;
Animal(string n) : name(n) {}
virtual void speak() {
cout << name << ": generic\n";
}
virtual ~Animal() {}
};
class Dog : public Animal {
public:
Dog(string n) : Animal(n) {}
void speak() override {
cout << name << ": Woof!\n";
}
};
class Cat : public Animal {
public:
Cat(string n) : Animal(n) {}
void speak() override {
cout << name << ": Meow!\n";
}
};
// Base POINTER -- works for any Animal subtype
void makeSpeak(Animal* a) {
a->speak(); // vtable dispatch
}
int main() {
Animal* a1 = new Dog("Rex");
Animal* a2 = new Cat("Whiskers");
makeSpeak(a1); // Rex: Woof!
makeSpeak(a2); // Whiskers: Meow!
// Heterogeneous array via base pointers
Animal* zoo[] = {a1, a2};
for (Animal* a : zoo) a->speak();
// SLICING -- bad:
// Animal sliced = *a1;
// sliced.speak(); // Animal::speak, NOT Dog!
delete a1; delete a2;
return 0;
}
```

Java

```
public class Main {
static class Animal {
String name;
Animal(String n) { name = n; }
```

```
public void speak() {
System.out.println(name+": generic");
}
}
static class Dog extends Animal {
Dog(String n) { super(n); }
public void speak() {
System.out.println(name+": Woof!");
}
}
static class Cat extends Animal {
Cat(String n) { super(n); }
public void speak() {
System.out.println(name+": Meow!");
}
}
// Base REFERENCE -- works for any subtype
static void makeSpeak(Animal a) {
a.speak(); // dynamic dispatch always on
}
public static void main(String[] args) {
Animal a1 = new Dog("Rex");
Animal a2 = new Cat("Whiskers");
makeSpeak(a1); // Rex: Woof!
makeSpeak(a2); // Whiskers: Meow!
// Heterogeneous array
Animal[] zoo = {a1, a2};
for (Animal a : zoo) a.speak();
// No slicing in Java
}
}
```

14. `const` Keyword (C++) vs `final` (Java)

C++'s **const** enforces immutability at variable, parameter, pointer, and member-function levels. Java's **final** covers variables, methods, and classes.

Usage	C++ (const)	Java (final)
Variable	`const int x = 5;`	`final int x = 5;`
Parameter	`void f(const int x)`	`void f(final int x)`
Member function	`void show() const {}`	`No direct equivalent`
Prevent override	`= 0 or avoid virtual`	`final void method() {}`
Class	`No keyword (use =delete pattern)`	`final class MyClass {}`
Pointer	`const int* p / int* const p`	`No raw pointers in Java`
Compile-time	`constexpr int x = 5*2;`	`Effectively final in lambdas`

C++

```
#include
using namespace std;
class Circle {
double radius;
public:
Circle(double r) : radius(r) {}
// const member fn -- cannot modify object
double getRadius() const { return radius; }
double area() const {
return 3.14159 * radius * radius;
}
void setRadius(double r) { radius = r; }
};
// const reference param -- no copy, no modify
void printArea(const Circle& c) {
// c.setRadius(10); // ERROR: const ref!
cout << "Area = " << c.area() << "\n";
}
constexpr int square(int x) { return x * x; }
int main() {
const Circle c(5.0);
// c.setRadius(10); // ERROR: const object
printArea(c);
constexpr int val = square(6); // compile-time: 36
cout << val << "\n";
// Pointer const varieties:
int x = 10, y = 20;
```

```
const int* p1 = &x; // pointer to const
int* const p2 = &x; // const pointer
// *p1 = 5; // ERROR -- value is const
p1 = &y; // OK -- pointer can move
// p2 = &y; // ERROR -- pointer is fixed
*p2 = 99; // OK -- value can change
return 0;
}
```

Java

```
public class Circle {
// final field -- set once, never changed
private final double radius;
public Circle(double r) {
this.radius = r;
}
// final method -- cannot be overridden
public final double getRadius() {
return radius;
}
public final double area() {
return Math.PI * radius * radius;
}
// final parameter -- cannot reassign inside
public static void printArea(final Circle c) {
// c = new Circle(10); // ERROR
System.out.println("Area = " + c.area());
}
}
// final class -- cannot be subclassed
final class ImmutablePoint {
final int x, y;
ImmutablePoint(int x, int y) {
this.x = x; this.y = y;
}
}
// class Extended extends ImmutablePoint {} // ERROR
class Main {
public static void main(String[] args) {
final Circle c = new Circle(5.0);
Circle.printArea(c);
int val = 36; // effectively final
Runnable r = () -> System.out.println(val);
r.run(); // 36
}
}
```

15. Access Specifiers in Inheritance

In C++, the inheritance mode changes what the **outside world** can see in the derived class. The derived class itself always has the same internal access — only external visibility is affected.

Base member	public inherit	protected inherit	private inherit
public	public	protected	private
protected	protected	protected	private
private	hidden	hidden	hidden

C++

```
#include
using namespace std;
class Base {
public:
int pub = 1;
protected:
int prot = 2;
private:
int priv = 3; // never reachable from derived
};
// PUBLIC (IS-A) -- most common
// pub stays public, prot stays protected
class PubDerived : public Base {
public:
void show() {
cout << pub << "\n"; // OK
cout << prot << "\n"; // OK (inside class)
// cout << priv; // ERROR always
}
};
// PROTECTED -- pub becomes protected
class ProtDerived : protected Base {
public:
void show() {
cout << pub << "\n"; // OK inside
cout << prot << "\n"; // OK inside
}
};
// PRIVATE (IMPLEMENTED-IN-TERMS-OF)
// everything becomes private
class PrivDerived : private Base {
public:
```

```
void show() {
cout << pub << "\n"; // OK inside
cout << prot << "\n"; // OK inside
}
};
int main() {
PubDerived pd;
pd.pub = 10; // OK -- still public
// pd.prot = 5; // ERROR: protected
ProtDerived rd;
// rd.pub = 10; // ERROR: now protected
PrivDerived vd;
// vd.pub = 10; // ERROR: now private
return 0;
}
```

Java

```
// Java only has ONE mode: public extends.
// Access is controlled by the member's OWN
// modifier in the parent class.
class Base {
public int pub = 1;
protected int prot = 2;
int pkg = 3; // package-private
private int priv = 4; // never inherited
}
class Derived extends Base {
public void show() {
System.out.println(pub); // OK
System.out.println(prot); // OK
System.out.println(pkg); // OK same pkg
// System.out.println(priv); // ERROR
}
}
class Main {
public static void main(String[] args) {
Derived d = new Derived();
System.out.println(d.pub); // OK
// d.prot: only in subclasses
// d.priv: never
d.show();
}
}
```

■ ***Java vs C++ Key Difference***	Java does not have protected or private inheritance. extends always behaves like C++ public inheritance. Private inheritance (implemented-in-terms-of) in Java is done via composition, not inheritance.

16. Shallow Copy vs Deep Copy

When an object contains a **pointer to heap memory**, copying it requires a decision: copy only the pointer address (shallow) or allocate new memory and copy the actual data (deep).

Shallow Copy (DANGEROUS)	Deep Copy (SAFE)
obj1.ptr --+ [data block] obj2.ptr --+ (shared! double-free risk)	obj1.ptr --> [data block A] obj2.ptr --> [data block B] (independent memory, safe)

• **Shallow copy**: both objects share the same heap block. When obj1 destructs and frees it, obj2 holds a dangling pointer. Accessing it is undefined behaviour (usually a crash).

• **Deep copy**: each object owns independent memory. Safe to destruct separately. Requires a custom copy constructor and copy assignment operator in C++.

C++

```
#include
#include
using namespace std;
// BAD: shallow (compiler default)
class ShallowStr {
public:
char* data;
ShallowStr(const char* s) {
data = new char[strlen(s)+1];
strcpy(data, s);
}
~ShallowStr() { delete[] data; }
// Default copy ctor copies the pointer VALUE
// -> both objects point to same memory
// -> double free on destruction: CRASH!
};
// GOOD: deep copy
class DeepStr {
public:
char* data;
DeepStr(const char* s) {
data = new char[strlen(s)+1];
strcpy(data, s);
}
// Deep copy constructor
DeepStr(const DeepStr& other) {
data = new char[strlen(other.data)+1];
strcpy(data, other.data);
```

```
cout << "Deep copy: " << data << "\n";
}
// Copy assignment operator
DeepStr& operator=(const DeepStr& other) {
if (this != &other;) {
delete[] data; // free old memory
data = new char[strlen(other.data)+1];
strcpy(data, other.data);
}
return *this;
}
~DeepStr() { delete[] data; }
};
int main() {
DeepStr s1("Mohid");
DeepStr s2 = s1; // deep copy ctor
DeepStr s3("test");
s3 = s1; // copy assignment
// s1, s2, s3 all own separate memory
return 0; // all three delete safely
}
```

Java

```
// Java: all objects are on the heap;
// variables hold REFERENCES, not raw ptrs.
// Shallow clone() shares mutable sub-objects.
class Student implements Cloneable {
String name; // Strings are immutable -- safe
int[] grades; // array is mutable -- risky!
Student(String name, int[] grades) {
this.name = name;
this.grades = grades;
}
// Shallow clone: grades array is SHARED
public Student shallowClone()
throws CloneNotSupportedException {
return (Student) super.clone();
}
// Deep clone: new independent array
public Student deepClone() {
return new Student(this.name,
this.grades.clone());
}
public static void main(String[] args) {
int[] g = {90, 85, 78};
Student s1 = new Student("Mohid", g);
```

```
Student s2 = s1.deepClone();
s2.grades[0] = 100; // does NOT affect s1
System.out.println(s1.grades[0]); // 90
System.out.println(s2.grades[0]); // 100
}
}
```

■ ***C++ Rule of Thumb*** In C++: if your class manages heap memory with new/delete, you MUST write a custom copy constructor, copy assignment operator, and destructor. The compiler-generated versions do shallow copy, leading to double-free crashes when two objects destruct.